



Why Apache Kafka?

Mohammad Fozouni (Ph.D.)

MLSECOPS IN ACTION COURSE



App 1

App 2

App 3

App 4

App 5

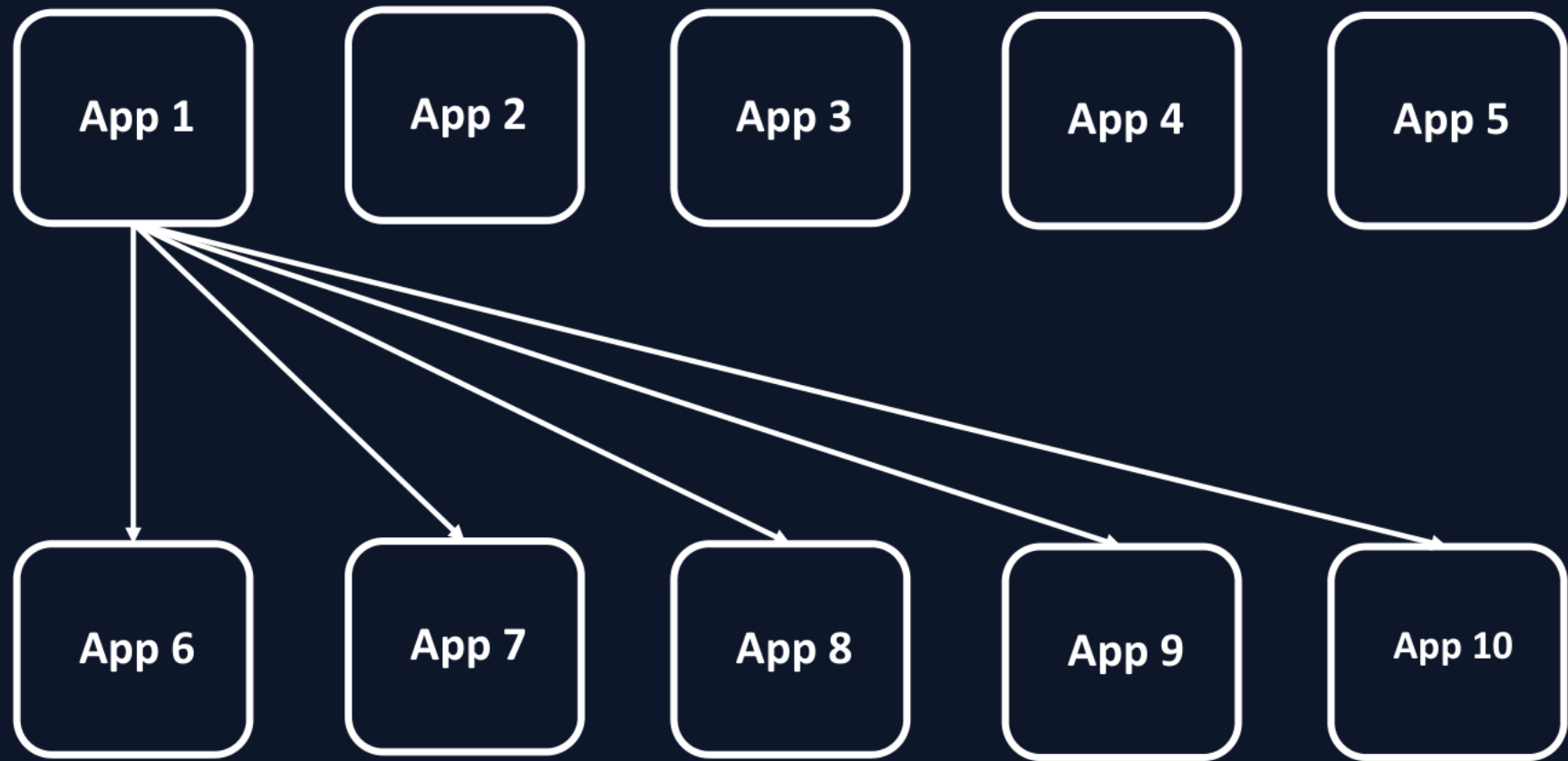
App 6

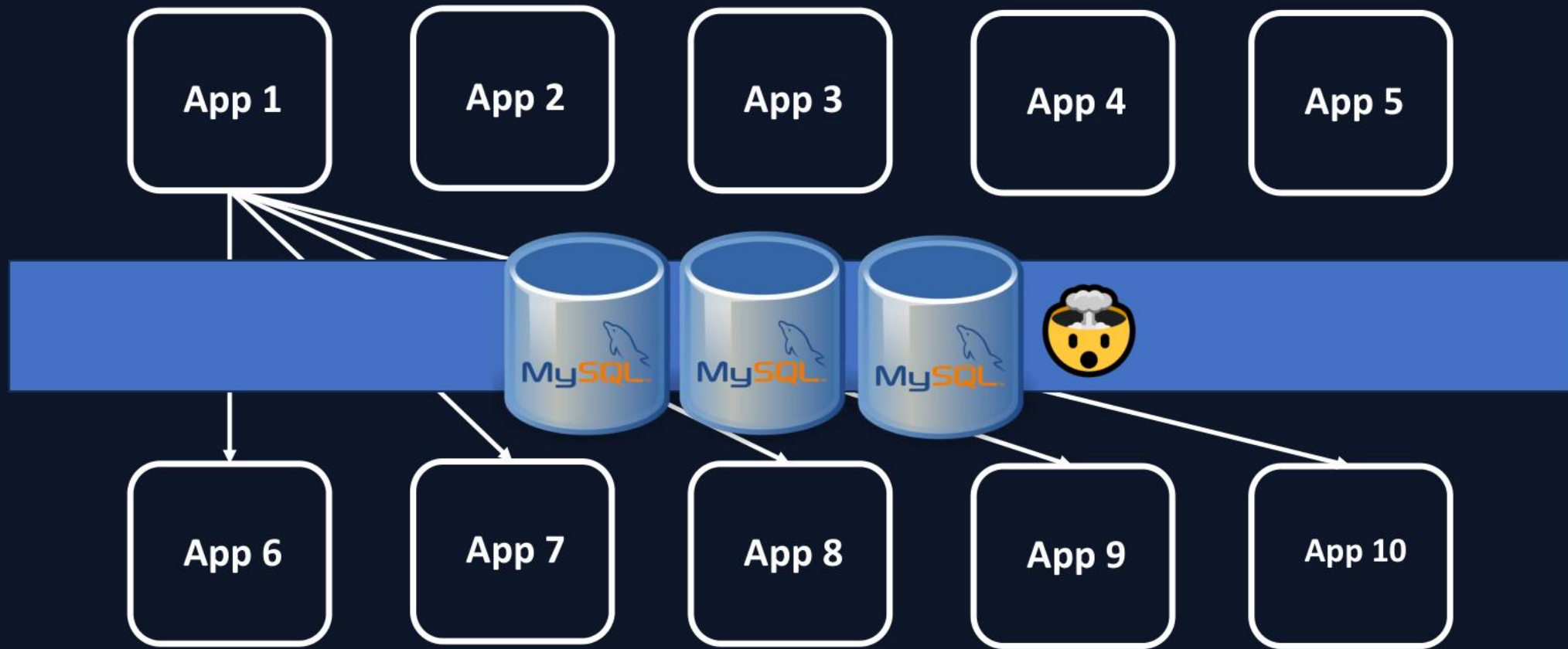
App 7

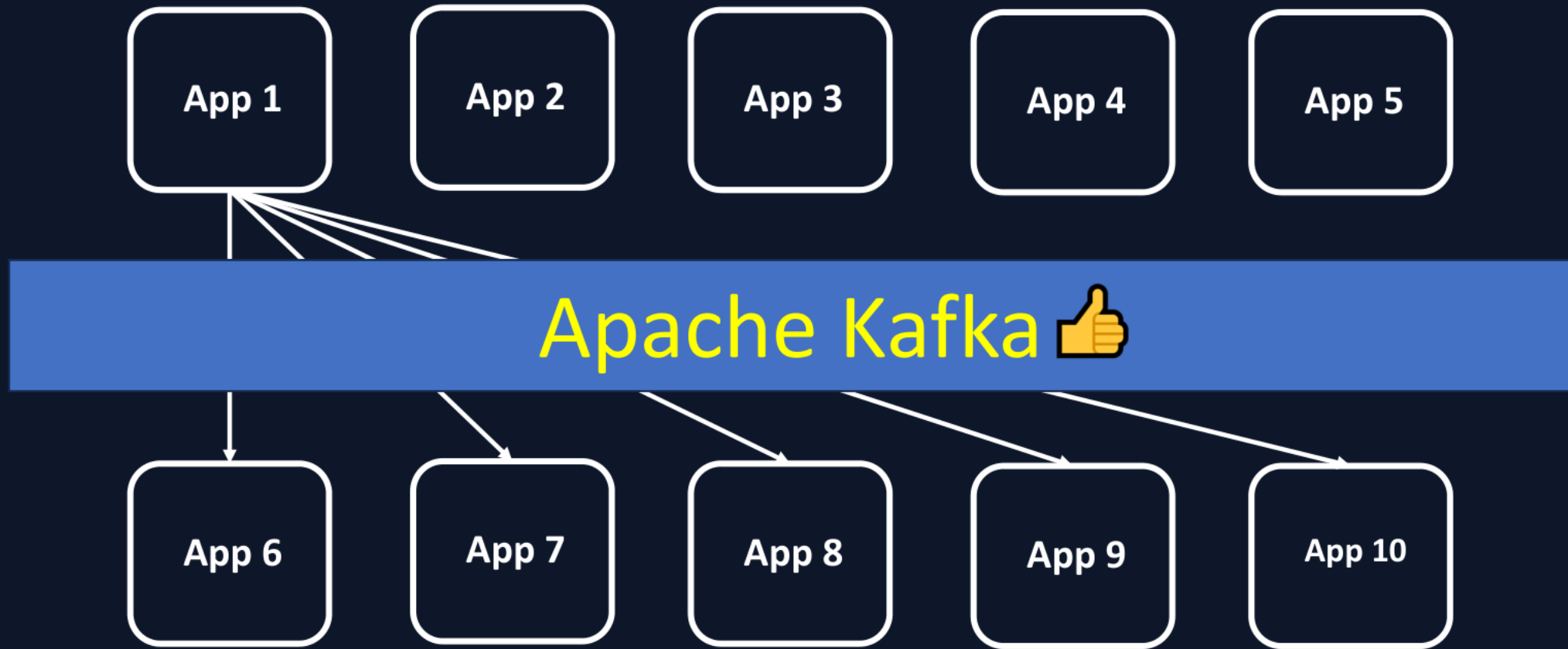
App 8

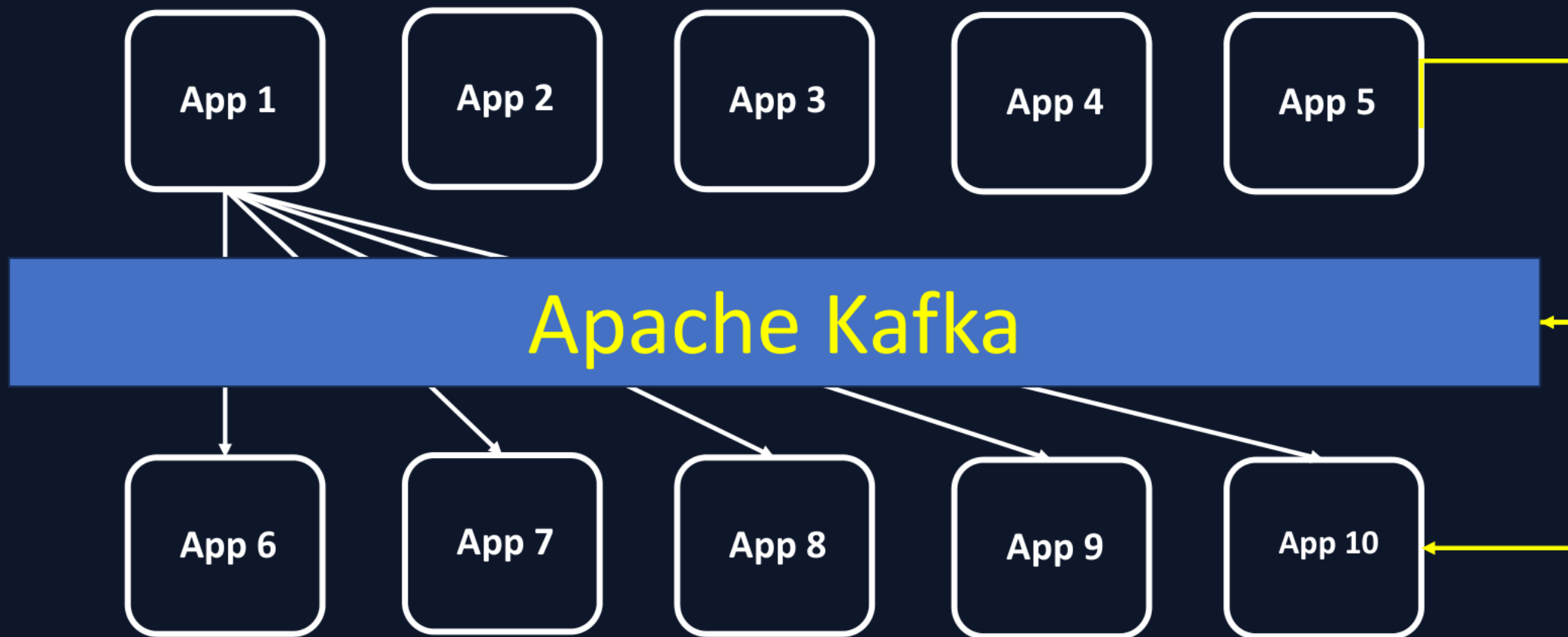
App 9

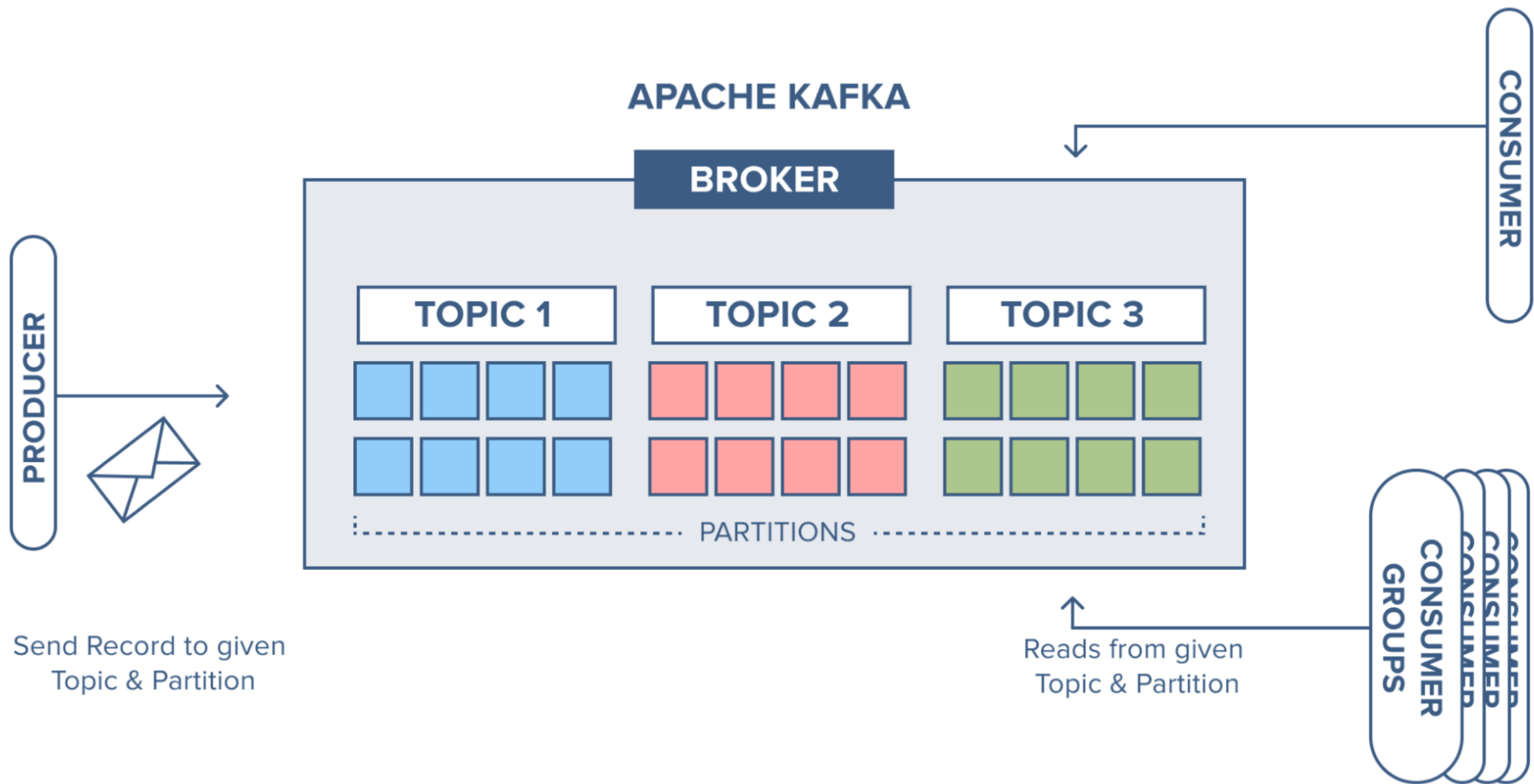
App 10

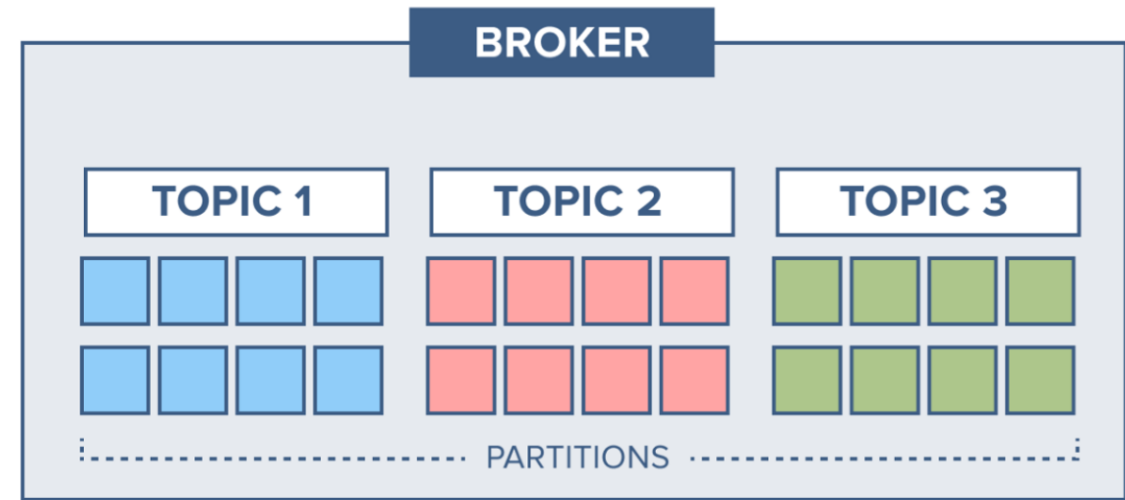












ارسال کننده چطوری تصمیم می‌گیره که پیام رو بفرسته سمت کافکا؟
یعنی به کدوم پارتیشن؟

Producer

Partitioning Strategies for Producers in Kafka



#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)

Partitioning Strategies for Producers in Kafka



#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)

Partitioning Strategies for Producers in Kafka

#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)



Partitioning Strategies for Producers in Kafka

#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)



Partitioning Strategies for Producers in Kafka

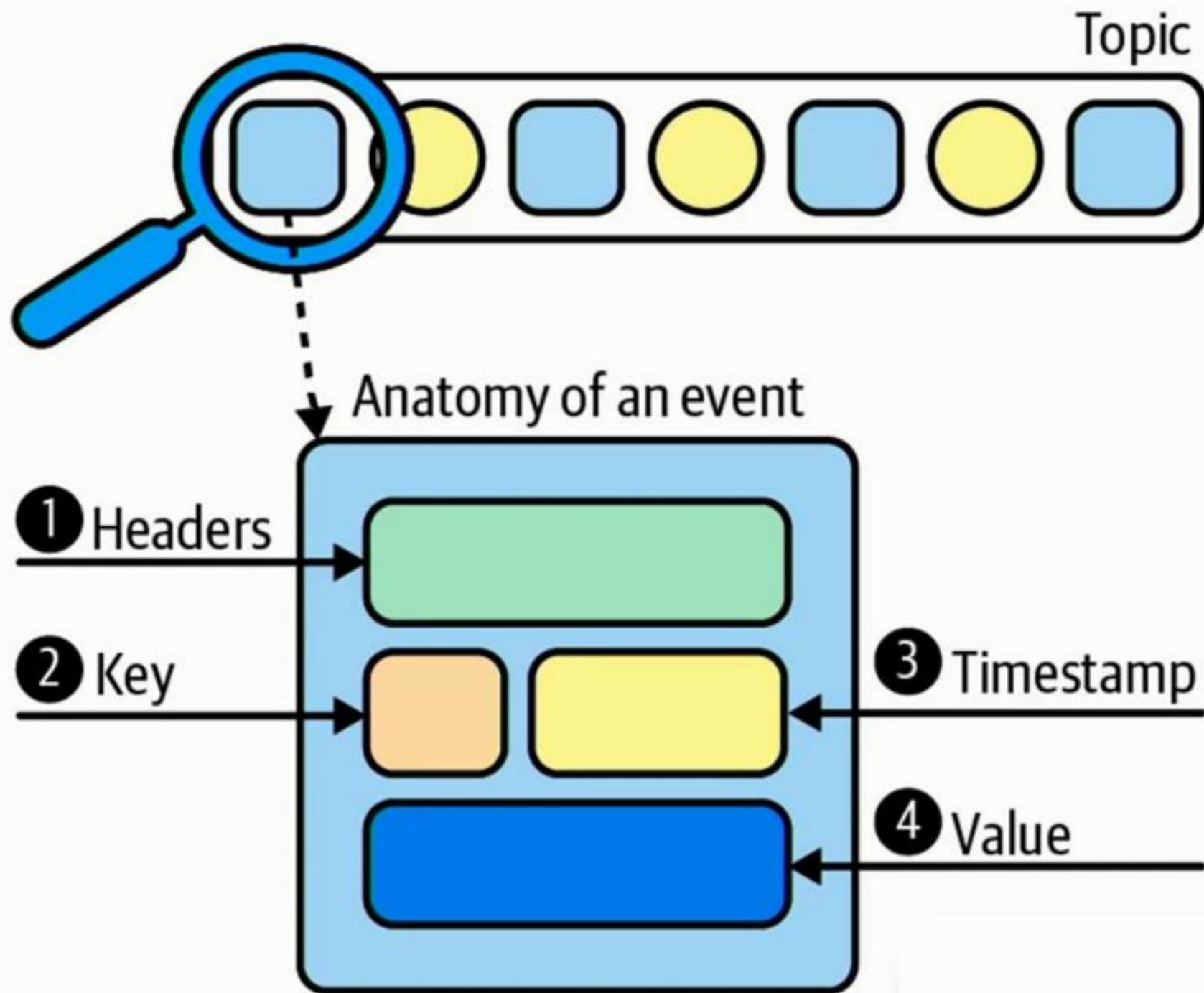
#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)

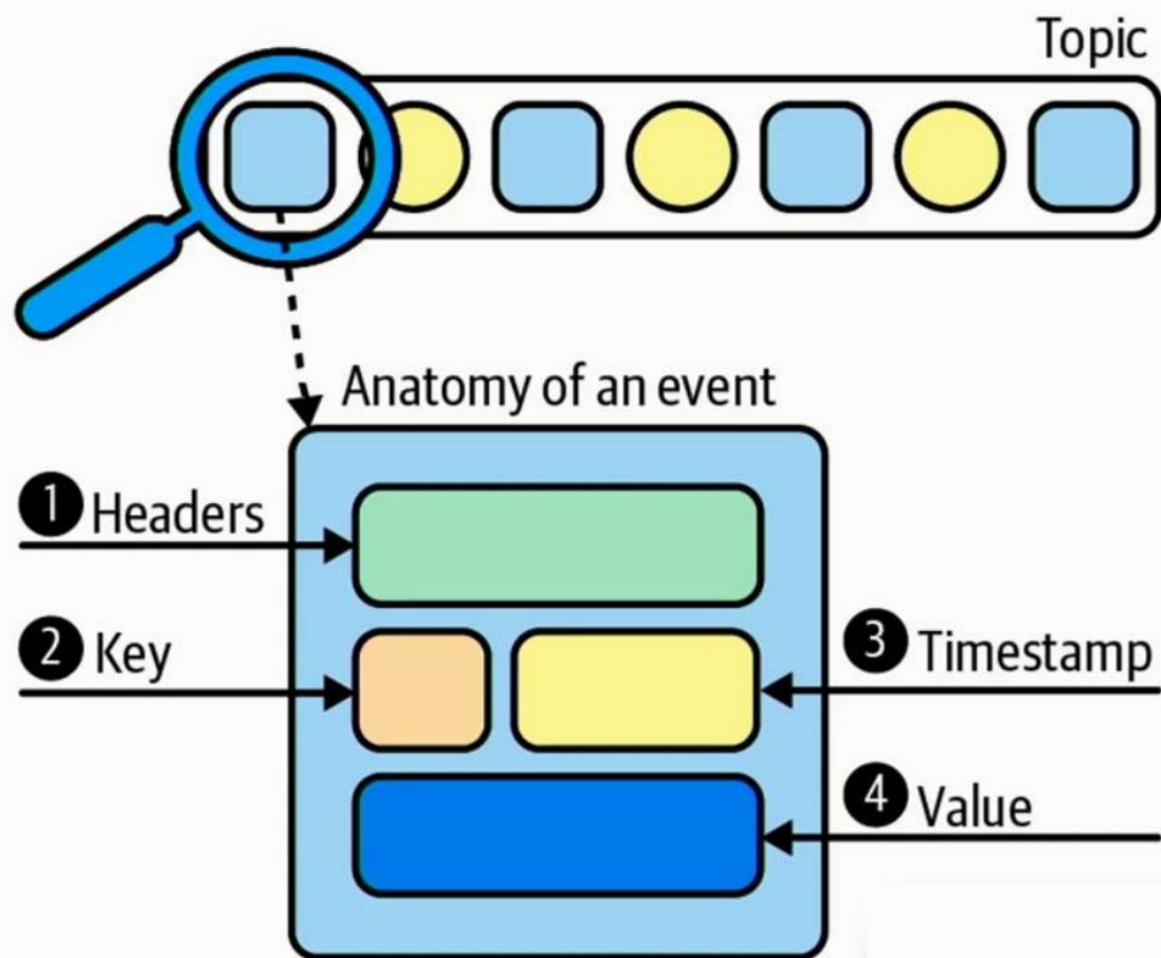


Partitioning Strategies for Producers in Kafka

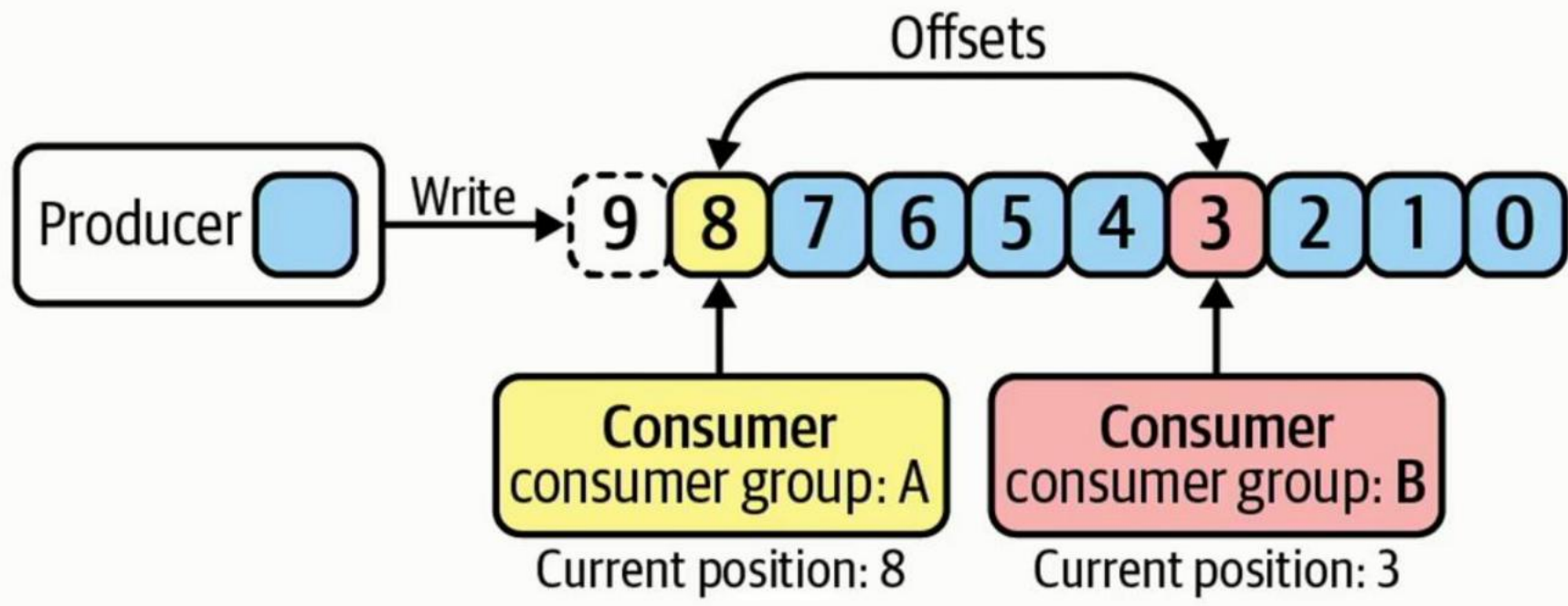
#	Strategy	How It Works	When Used
1	Explicit	You specify partition number directly in code	You know exactly where to send the message
2	Key-based (Hash)	$\text{hash}(\text{key}) \% \text{numPartitions}$	You provide a key (same key = same partition)
3	Sticky	Stick to one partition, batch messages, then switch to another	No key and no partition (default since Kafka 2.4+)
4	Round-robin	Messages distributed evenly, one partition at a time in rotation	No key and no partition (legacy, older Kafka)
5	Custom	Your own logic (implement Partitioner interface)	You need custom business logic (e.g., by geography, order value)
6	Default Partitioner	Uses Sticky (Kafka 2.4+) or Round-robin (older)	No key, no explicit partition (internal default)

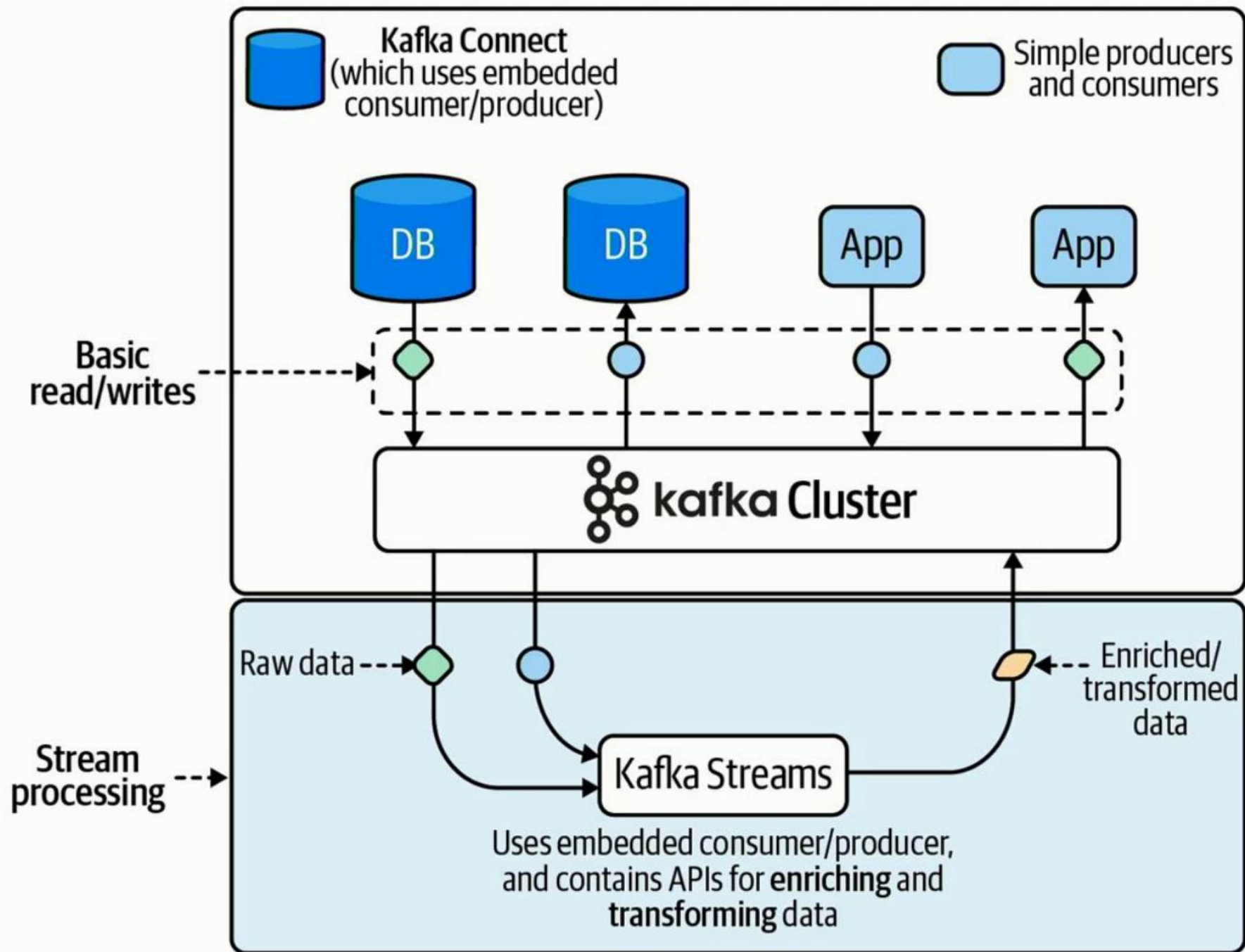


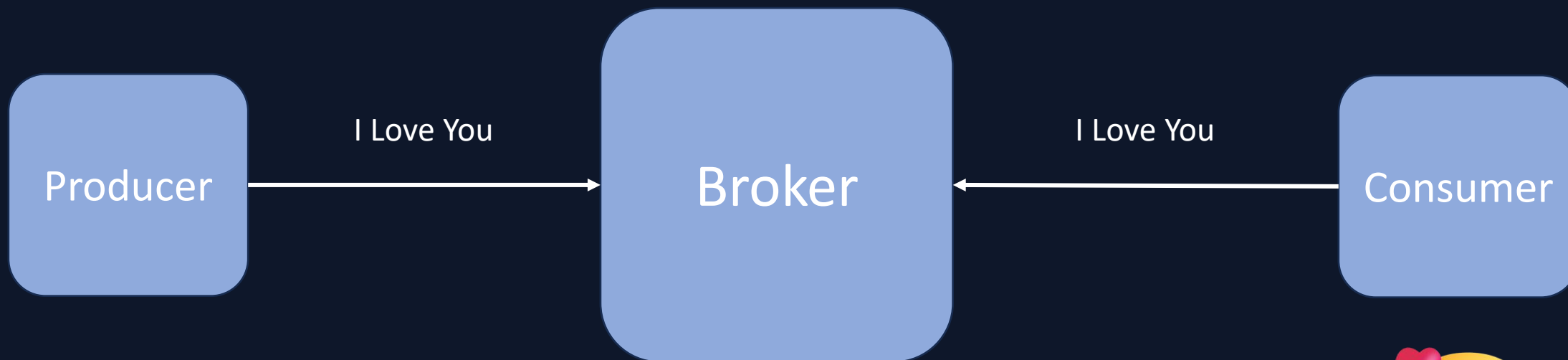




- **Key:** Optional (used for partitioning)
- **Value:** Mandatory (contains the message content)
- **Timestamp:** Optional (time of production)
- **Headers:** Optional (additional metadata in key-value pairs)







Broker

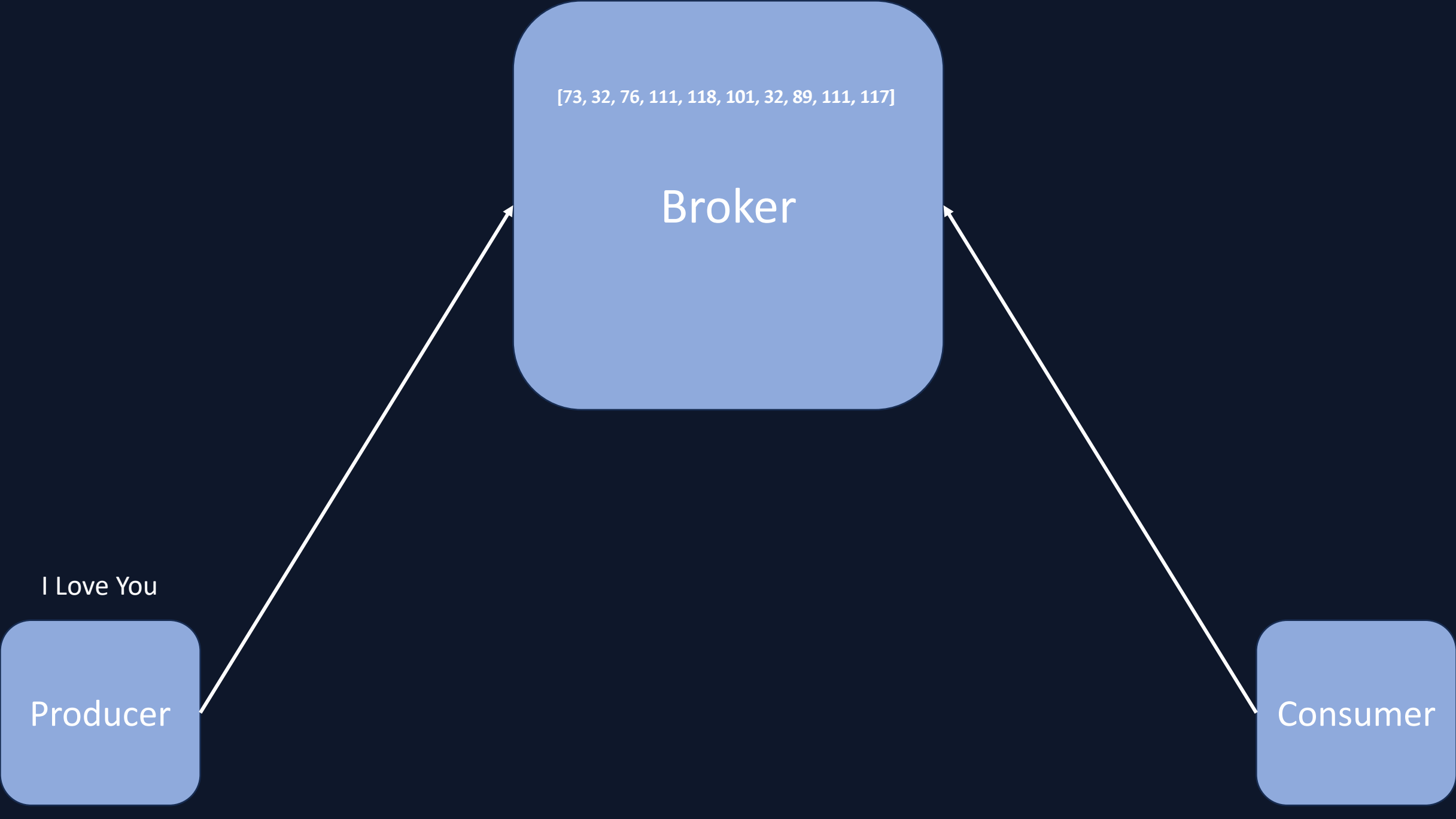
Producer

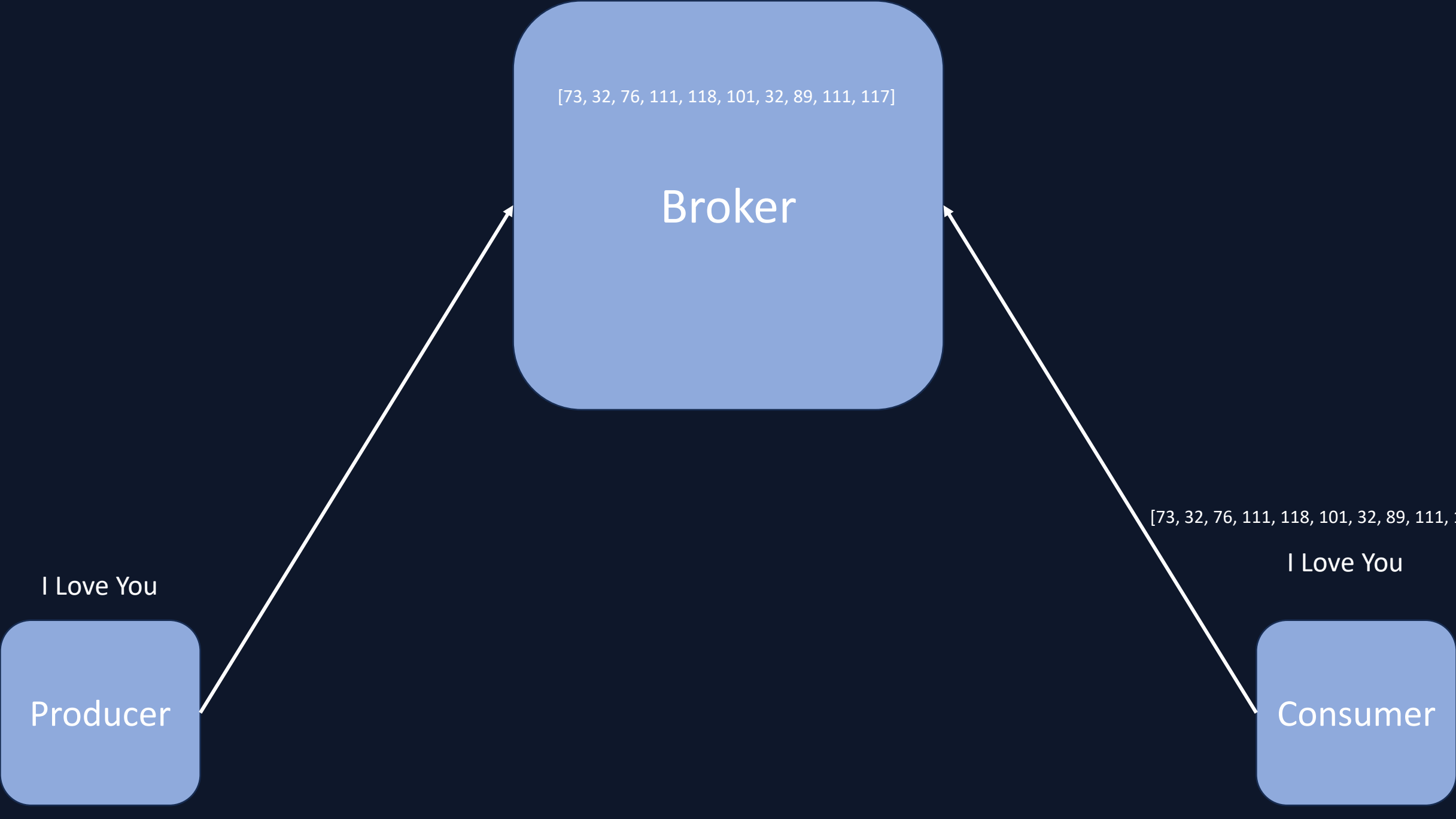
Consumer

I Love You

[73, 32, 76, 111, 118, 101, 32, 89, 111, 117]







I Love You

String

[73, 32, 76, 111, 118, 101, 32, 89, 111, 117] ==> 10 bytes

{"message": "I Love You"}

JSON

[123, 34, 109, 101, 115, 115, 97, 103, 101, 34, 58, 32, 34, 73, 32, 76, 111, 118, 101, 32, 89, 111, 117, 34, 125] ==> 25 bytes

I Love You

String

[73, 32, 76, 111, 118, 101, 32, 89, 111, 117] ==> 10 bytes

{"message": "I Love You"}

JSON

[123, 34, 109, 101, 115, 115, 97, 103, 101, 34, 58, 32, 34, 73, 32, 76, 111, 118, 101, 32, 89, 111, 117, 34, 125] ==> 25 bytes

Why does Kafka use byte arrays?

1. **Language-neutral** — any programming language can read bytes.
2. **Fast** — the broker doesn't waste time understanding or transforming data.
3. **Flexible** — you can serialize anything (text, binary, Avro, Protobuf, etc.) as long as producer and consumer agree on the format.

Real-World Examples of Using Kafka

Event-Driven (Reactive)

User signs up. Our service sends welcome email, creates profile, adds to CRM

Kafka role: Event bus between microservices

Data Integration (Batch/Streaming)

MySQL changes ==> Kafka ==> data warehouse

Kafka role: Pipeline for syncing data between systems

Stream Processing (Analytics)

Clickstream data ==> Real-time dashboard showing active users

Kafka role: Source of streaming data for processing

Log Aggregation (Monitoring)

All app logs ==> Kafka ==> Elasticsearch for searching

Kafka role: Centralized log collector

What we will see in the next sessions?

In the next video we see a Hello-World Demo

Clustering Kafka

(How we can set up
some VMs that work
together)

Quix for Python-native Streaming

(How we can process
data as soon as possible
by python codes)

ksqlDB

(Process real time data
with SQL)

Security in Kafka Deployments

(ACLs & Encryption)

حالا بریم برای انجام کار عملی
نصب کافکا و موارد جانبی

